

POLITECNICO
MILANO 1863

# Best Bike Paths

## Design Document (DD)

A. Rubino E. Uvinsi I. Vinciguerra

Academic Year 2025/2026

# Contents


**1 INTRODUCTION** **3**
- 1.1 Scope . . . 3
- 1.2 Definitions, Acronyms, Abbreviations . . . 3
  - 1.2.1 Definitions . . . 3
  - 1.2.2 Acronyms . . . 4
- 1.3 Revision History . . . 4
- 1.4 Document Structure . . . 4

**2 ARCHITECTURAL DESIGN** **6**
- 2.1 Overview: High level components and their interaction . . . 6
  - 2.1.1 System View . . . 6
  - 2.1.2 Detailed View . . . 7
- 2.2 Component view . . . 8
- 2.3 Deployment view . . . 11
- 2.4 Runtime view . . . 13
- 2.5 Component interfaces . . . 25
  - 2.5.1 Client subsystem . . . 25
  - 2.5.2 Backend system . . . 27
  - 2.5.3 Data system . . . 28
- 2.6 Selected architectural styles and patterns . . . 28
- 2.7 Other design decisions . . . 30

**3 USER INTERFACE DESIGN** **32**

**4 REQUIREMENTS TRACEABILITY** **37**

**5 IMPLEMENTATION, INTEGRATION AND TESTING** **40**
- 5.1 Overview . . . 40
- 5.2 Implementation Plan . . . 40
- 5.3 Component Integration and Testing . . . 41
  - 5.3.1 Testing Offline Functionalities . . . 41
  - 5.3.2 Testing Online Functionalities . . . 43
- 5.4 Additional Specifications on Testing . . . 47

**6 EFFORT SPENT** **48**

**7 REFERENCES** **48**

# 1 INTRODUCTION

## 1.1 Scope

The core function of the BBP system is to provide a comprehensive, community-driven platform to all types of cyclists. The system acts as an intermediary, processing heterogeneous data inputs from various sources, including external providers, and synthesizing this information to generate reliable output for all users.
The architectural design detailed in this document aims to fulfill the requirements outlined in the RASD document. The BBP system is designed to be scalable, modular, and maintainable, ensuring that it can adapt to future needs and technological advancements.
The design is centered around **a multi-tier client-server architecture** that separates concerns across different layers, promoting a clear division of responsibilities. This approach enhances the system's ability to handle diverse data sources and user interactions effectively.
The BBP system incorporates various components, each responsible for specific functionalities such as user management, data acquisition, path reconstruction, and path publication. These components interact seamlessly to provide a cohesive user experience while ensuring data integrity and reliability.

## 1.2 Definitions, Acronyms, Abbreviations

### 1.2.1 Definitions

- **Global Positioning System**: a satellite-based navigation system owned by the U.S. government that uses a constellation of orbiting satellites, ground control stations, and receivers to provide precise positioning, navigation, and timing information.
- **Accelerometer**: a sensor that measures linear acceleration (changes in velocity over time) along one or more axes.
- **Gyroscope**: a sensor that measures angular velocity. It detects how fast an object is rotating around a specific axis, which is essential for determining a device's current orientation in space.
- **Meteorological Service Provider**: a formal entity that is responsible for the collection, processing, analysis, forecasting, and dissemination of meteorological and related environmental information and services.
- **Map Service Provider**: a formal entity that provides geospatial infrastructure, data, and rendering capabilities to software applications. It manages the acquisition, storage, and processing of raw Geographic Information Systems (GIS) data such as satellite imagery, vector topology, and traffic signals, allowing client applications to request processed map

assets and perform spatial calculations on demand, without maintaining a proprietary geospatial backend.

### 1.2.2 Acronyms

- **BBP**: Best Bike Paths
- **RU**: Registered User
- **RASD**: Requirements Analysis and Specification Document
- **DD**: Design Document
- **GPS**: Global Positioning System
- **API**: Application Programming Interface
- **UI**: User Interface
- **JSON**: JavaScript Object Notation
- **REST**: Representational State Transfer
- **HTTP**: HyperText Transfer Protocol
- **HTTPS**: HyperText Transfer Protocol Secure
- **SQL**: Structured Query Language
- **NoSQL**: Not Only SQL
- **GIS**: Geographic Information Systems
- **XML**: eXtensible Markup Language

## 1.3 Revision History

## 1.4 Document Structure

1. **Introduction**: This section aims to give a brief description of the project. In particular it offers an overview of the purpose of the document and a very general description of the adopted solution.
2. **Architectural Design**: This part of the document describes at different levels of granularity how the architecture is designed.
3. **User Interface Design**: This section shows the wire frame that constitutes the platform UI.
4. **Requirements Traceability**: This section contains the traceability matrix that maps the requirements to the components that implement them.

5. **Implementation, Integration and Test Plan**: This section provides a description of the implementation integration and testing details, useful for the developers and producer of the platform.

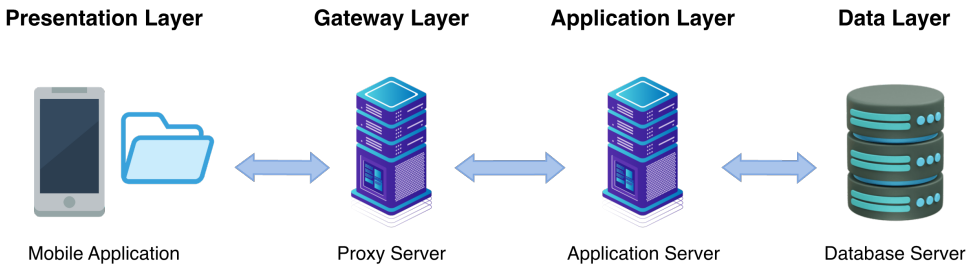
6. **Effort Spent**: It contains a table that shows the effort spent to write this document.

7. **References**: It contains the references to any documents and software used to write this document.

# 2 ARCHITECTURAL DESIGN

## 2.1 Overview: High level components and their interaction

The following diagram provides a high-level overview of the main components of the BBP system and their interactions.




### 2.1.1 System View

The architectural approach we adopted for the BBP platform is a 4-tier architecture, which consists of the following layers:

- **Presentation Layer**: this layer is responsible for the user interface and user experience. It includes a mobile application that allow users to interact with the platform while interacting with the internal storage of the mobile device to store the user's path inventory.
- **Gateway Layer**: this layer acts as an intermediary between the presentation layer and the application layer. It handles requests from the user interface and forwards them to the appropriate services in the application layer.
- **Application Layer**: this layer contains the core logic of the platform.
- **Data Layer**: this layer is responsible for data storage and management. It includes databases that store user information, path data, and other relevant information.

As specified above, the application layer is divided into two sub-layers to separate the main application services that implement main functionalities of the application from the user management service that handles user authentication, authorization, and profile management. When the Gateway Layer receives a request from the Presentation Layer, it forwards the request to the appropriate service in the Application Layer. The service processes the request and interacts with the Data Layer as needed to retrieve or store data. Once the service has completed its task, it sends a response back to the Gateway Layer, which then

forwards the response to the Presentation Layer for display to the user. Furthermore, our application strongly depends on external services, mainly to retrieve meteorological data and to process and display paths from raw sensor data. The communication protocol that is used in these scenarios are Rest API calls. In an ideal implementation, our architecture would be geographically distributed, with multiple instances of each layer deployed in different regions to ensure high availability and low latency for users around the world. The geographical distribution would also contribute to reduce latency when communicating with external services, as instances could be deployed closer to the external services' servers.

### 2.1.2 Detailed View

**Client Tier**: The client tier contains the mobile application that provides the UI for the BBP platform to users to allow them to interact with the platform. Through the UI, the users can perform various actions that can be grouped into three main categories: account management, path insertion, and path research. The last category is the only one that is also available to non-registered users.

- **Account Management**: this functionality allows users to create and manage their accounts on the platform. Users can register, log in, update their profile information, and change their searching area.
- **Path Insertion and Publication**: this functionality allows registered users to insert new bike paths into the platform. Users can either manually input path data or use the automated path data collection feature to record their bike rides using the mobile device's sensors.
- **Path Research**: this functionality allows users to search for bike paths that connect their desidered origin and destination points. When they exist, users can customize the visualization of the existing paths and they can select a path to view more detailed information about it.

While account management, path processing, publication and research require an internet connection, the path insertion functionality is partially available offline. This is due to the fact that there is a subtle yet fundamental difference between the concepts of "bike path" and "record": a record contains a bike path and adds a set of performance metrics and meteorological data that are only available to the user that recorded it, while a bike path is an entity that can be made available to all users on the platform and it contains the "publishable" data of the cycling sessions: the geographical reconstruction of the path, the status of its streets and the presence of obstacles. Therefore, since a record is private, it is only stored locally on the user's device and it can be accessed offline. Since the record entity contains a bike path, its creation still does need an internet connection in order to interact with the external providers to retrieve meteorological data and to process raw sensor data into a human-readable and visualizable format.

The same concept applies to manually inserted paths that have not been published yet: we handle these drafts locally on the user's device until (and if ever) the user decides to publish them.
The presentation layer is therefore responsible both for providing the user interface and for managing the local storage of the user's path inventory.

With this structure for these fundamental entities, we manage to obtain a smooth and easy way to handle private and public data: records already have the publishable information within the bike path that they contain creating a clear separation between private and public data. When a user decides to publish a path, the application simply extracts the bike path from the record and sends it to the server for publication.

**Gateway Tier**: The gateway tier acts as an intermediary between the presentation layer and the application layer. It receives requests from the mobile application and forwards them to the appropriate services in the application tier. In particular, the gateway tier separates account management and permission related requests to a specific server, whereas all path related requests are forwarded to the application server.

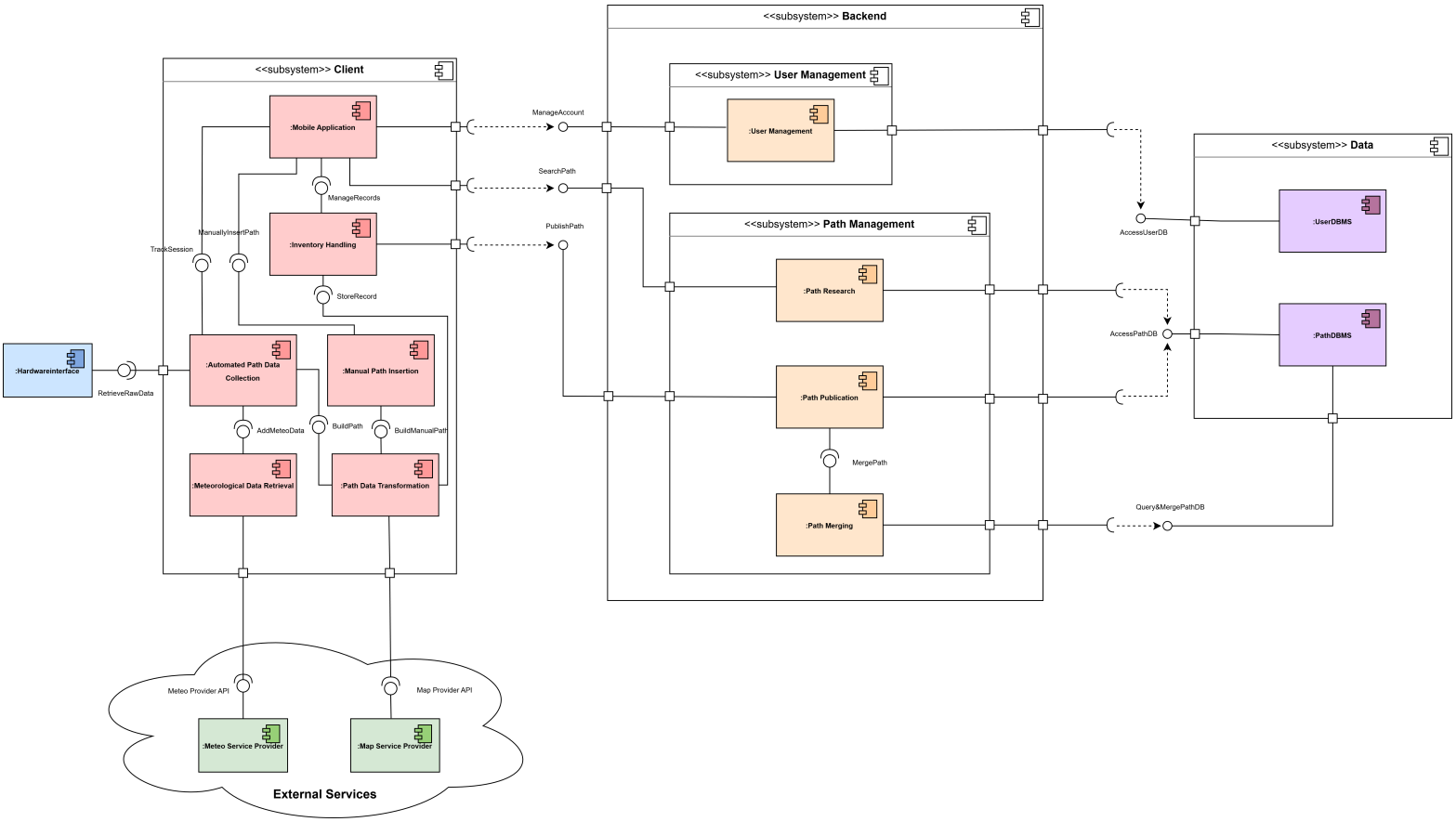
**Application Tier**: The application tier contains the core logic of the BBP platform and it is responsible for processing both path publication and research requests. It also contains the authentication service that manages user accounts and permissions. The application server in this tier is structured to be scalable and it could therefore be replicated and distributed geographically.

**Data Tier**: The data tier is responsible for storing and managing all the data used by the BBP platform. It contains two main databases: one for user account information and another for published bike paths. Both databases are designed to be highly available and to provide eventual consistency across all replicas, ensuring that all users have access to the most up-to-date information.

In the next sections we go deeper in the details of the components and the deployment.

## 2.2 Component view

This section gives a more detailed description of the components of the architecture of the BBP system.



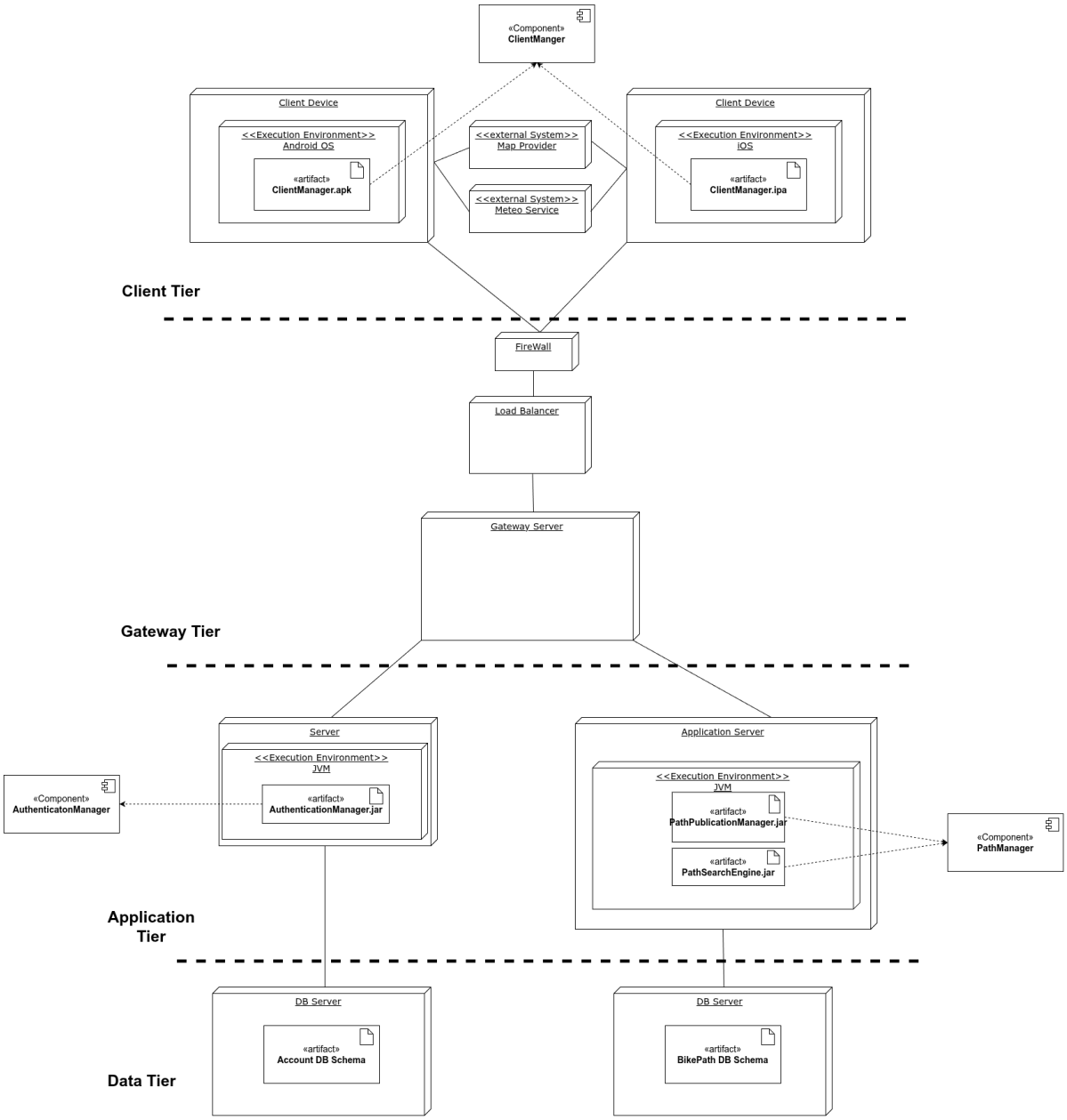

The main components of the system are:

- **User Management Service**: This service is responsible for managing user accounts, including registration, authentication, and profile management. It ensures that only authorized users can access certain features of the application.
- **Automated Path Data Collection Service**: This service collects raw data from the mobile device's sensors during a cycling session. It gathers information such as GPS coordinates, speed, elevation, and other relevant metrics and stores them in the local storage of the device.
- **Path Data Transformation Service**: This service processes the raw sensor data collected during a cycling session and transforms it into a human-readable and visualizable format. Apart from reconstructing paths, it gives an evaluation of each street's conditions and identifies obstacles along the route based on sensor data. It is also responsible for computing and storing performance metrics.

- **Meteorological Data Retrieval Service**: This service interacts with external meteorological data providers to retrieve weather information for the area where the cycling session took place. It ensures that the weather data is accurately associated with the corresponding record data.
- **Path Research Service**: This service allows users to search for bike paths that connect their desired origin and destination points. It retrieves relevant paths from the database and provides customization options for visualization.
- **Path Publication Service**: This service handles the publication of bike paths to the platform. It ensures that the path data is properly formatted and stored in the database, making it accessible to other users.
- **Manual Path Insertion Service**: This service allows users to manually input bike path data into the platform. It provides an interface for users to enter path details and ensures that the data is correctly formatted and stored.
- **Inventory Handling Service**: This service manages the local storage of the user's path inventory on the mobile device. It allows users to access and manage their records and drafts offline.
- **Path Merging Service**: This service is responsible for merging newly published paths with existing ones in the database. It ensures that the aggregated data reflects the combined inputs from multiple users, updating street conditions and obstacle information accordingly. In particular, it recalculates the status of each street segment based on the new data provided by users so that, in the end, the same street segment will have a single community-agreed status and list of obstacles that exists within the database.

## 2.3 Deployment view

This chapter describes the deployment view of the BBP system. it outlines the execution environment of the system and details the gegraphical distribution of the hardware components that host the software making up the application.



The deployment architecture follows a Four-Tier approach.

- **The first tier** is a Thick Client able to provide parts of the functionalities (gathering of information and offline storage) also when it is not connected to the Server.
- **The second tier** consists of the Gateway and the Load balancer. The Gateway exposes the public endpoint and route the requests toward the correct type of service (Account Server or Application Server). This layer works also as shield against external treats. The communication between first and second tier takes place via HTTPS protocol.
- **The third tier** consists of two types of servers: The ones to manage everything that is related to the personal account of a user and the ones that regard the "shared" part of the application, i.e. the publication of bike paths. This separation between the two aspects allows to easily achieve independent scalability of the two types of service, and allow the system to avoid the congestion of both services if one is managing a big quantity of requests. In other words, this kind of separation introduces a high level of decoupling.
- **The fourth and last tier** is dedicated to the Data, the databases do not communicate directly with internet but only with the Servers responsible to manage the information. The communication between these tiers takes place via TCP/IP protocol.

We proceed to provide further details about the elements in the graph.

- **Database Server for Published Paths**
  This database stores all the published paths by registered users. The system shall be highly available and shall be able to provide eventual consistency across all the replicas, so all the set of available paths will be visible to all the users around the world.
- **Database Server for Registered Users**
  This Database stores all the personal information of registered users such as name, e-mail and the hashed password.
- **Application Server**
  This Server is responsible to process all the requests regarding the publication and the search of bike path from the final user. This component manipulates data, and then it has direct access to the related Database.
- **Account Server**
  This Server is responsible of managing all the requests that regard the accounts of the end user and the related information. This component manipulates personal data of the users and it has access to the related Database.

- **Load Balancer**
  It distributes incoming requests across multiple Gateway to optimize system performances and avoid overloading one server.

- **Firewall**
  Monitors incoming network traffic, blocking potentially harmful packets before they reach the system. It creates a safe perimeter around the network by being positioned before the load balancers.

- **Mobile Device**
  This represents the device used by the end user. It requires the app to be installed in order to access the BBP application.

## 2.4 Runtime view

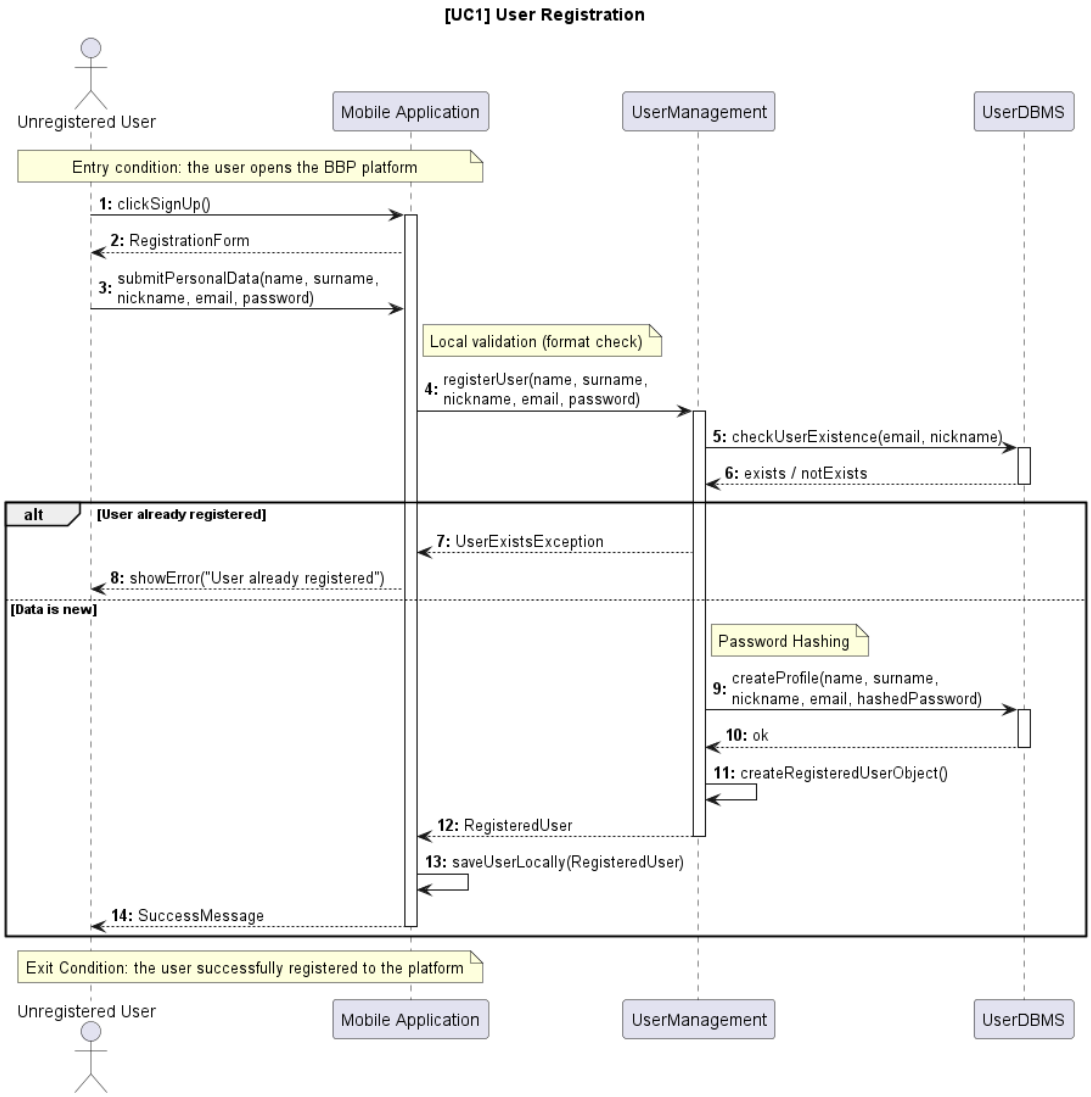
In this section, we describe the dynamic behavior of the BBP system by detailing the flow of messages between the architectural components. We utilize Sequence Diagrams to model the interactions required to fulfill the primary Use Cases as described in the RASD. To ensure the diagrams remain readable and focused on the core logical flows, we applied the following abstractions:

- **Authorization Checks:** For the sake of simplicity, we have omitted the repetitive verification steps where the system ensures a request is authorized. It is implicitly assumed that the User Management Service and the Gateway enforce security policies (e.g., ensuring that a user can only view or modify their own private sessions and cannot alter data belonging to other users).

- **Error Handling:** We have excluded low-level error handling related to incorrectly specified parameters, network timeouts, or minor validation issues, unless they are significant to the specific use case (e.g., login failure).

- **Caching and Optimization:** While components such as the Load Balancer or API Gateway may implement caching mechanisms to reduce latency and database load, our diagrams represent the general case where requests are processed by the application servers and reach the persistence layer.

These abstractions allow us to focus on the main architectural interactions and the data flow necessary to execute the functional requirements.



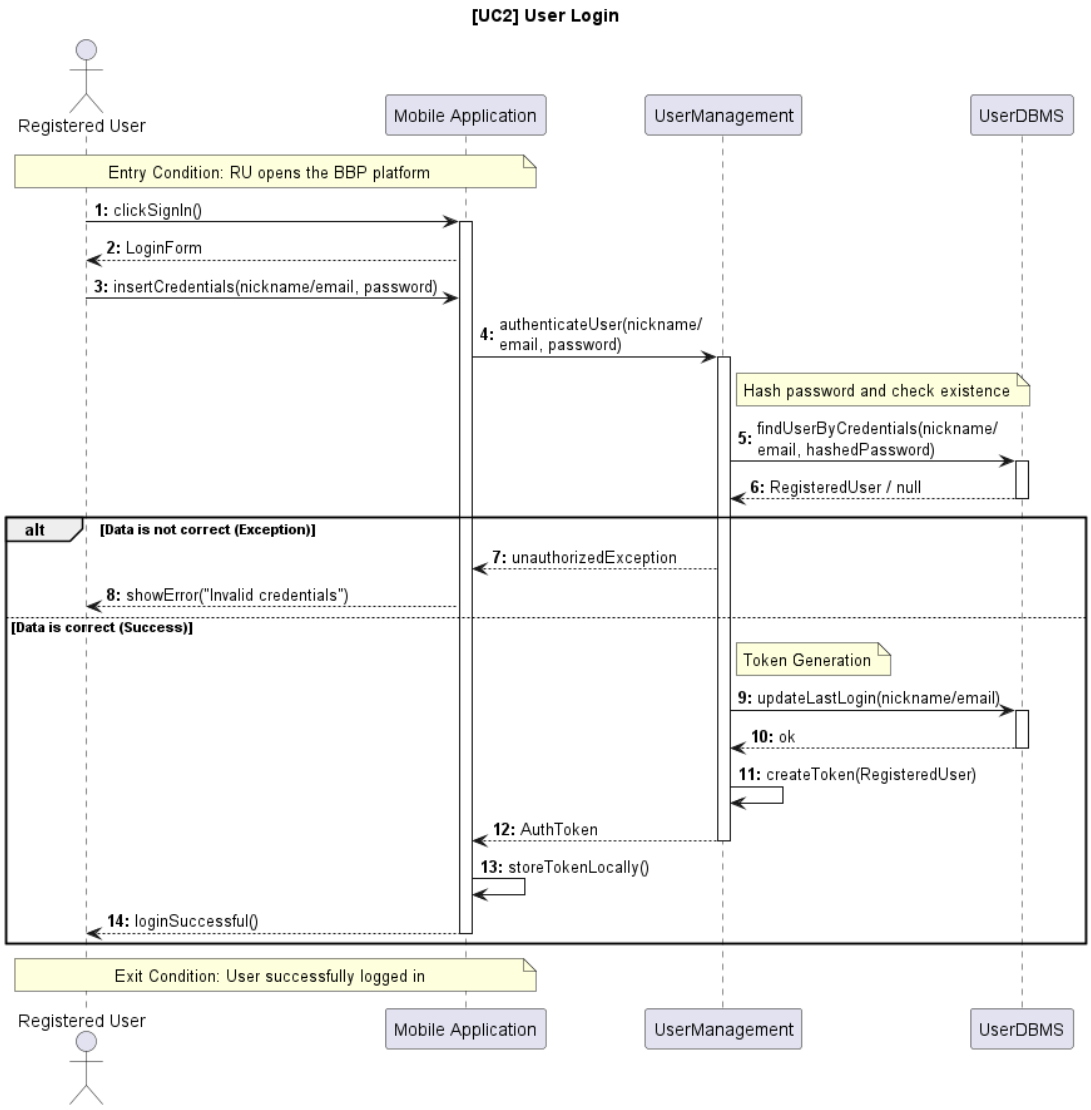

This diagram depicts the registration process initiated by the Unregistered User. Upon submitting personal data via the Mobile Application, the request is forwarded to the UserManagement component.

The system first queries the UserDBMS to verify that the email or nickname is not already in use.

If the user is already registered: The flow terminates with an exception, and an

error message is displayed to the user.

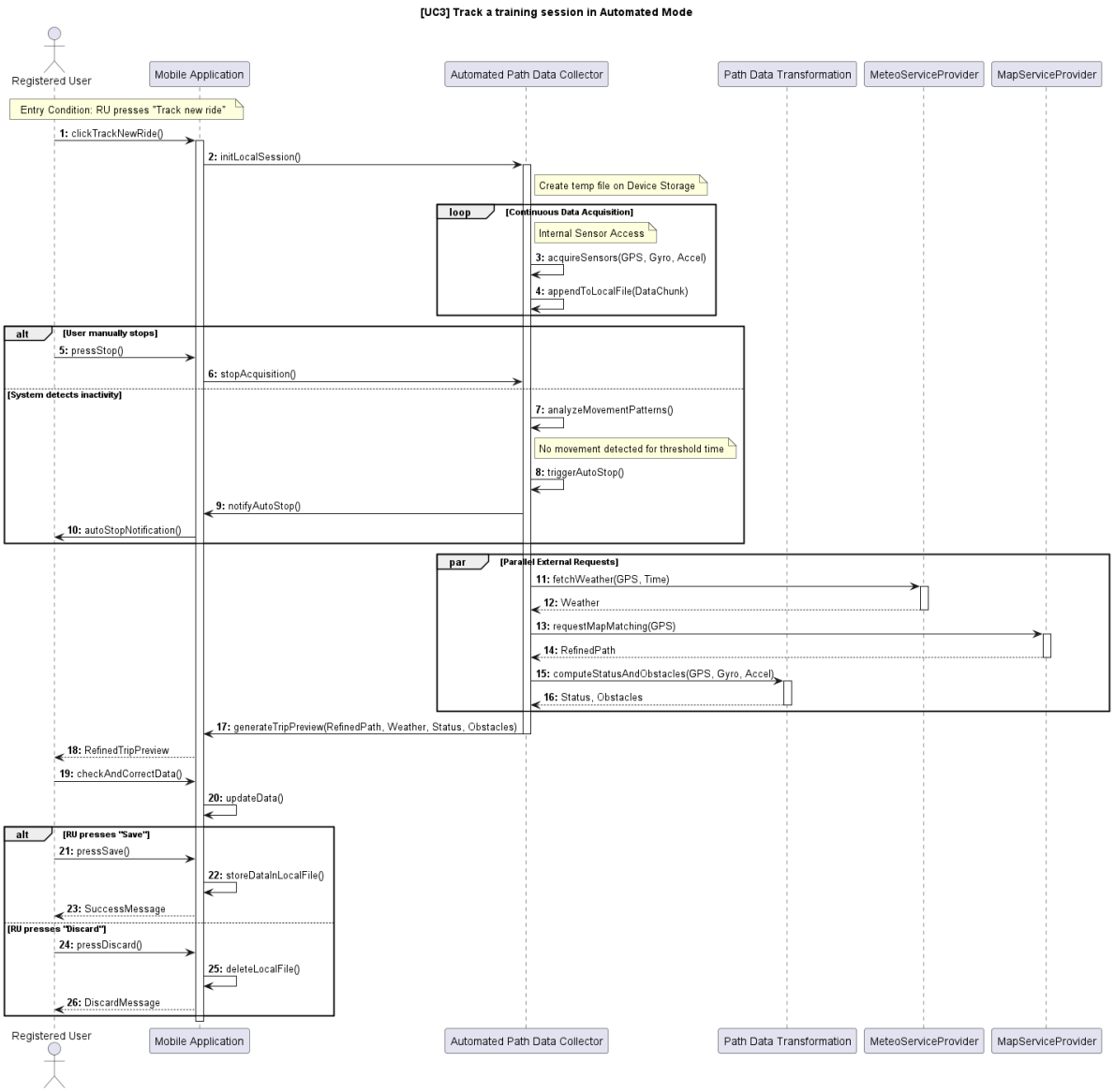
If the user is not registered yet: The UserManagement component hashes the password and persists the new profile in the database. Finally, it returns a confirmation to the Mobile Application, which saves the user's session locally to complete the process.



This diagram illustrates the authentication flow initiated by a Registered User. After the user enters their credentials, the Mobile Application forwards the authentication request to the UserManagement component.

The system verifies the user's identity by hashing the provided password and querying the UserDBMS for a match.

If the data is incorrect, the database returns no result, causing the system to raise an exception and display an "Invalid credentials" error to the user. If the data is correct, the UserManagement component updates the user's login history in the database and generates a secure session token (AuthToken). This token is returned to the Mobile Application, which stores it locally to maintain the authenticated session.

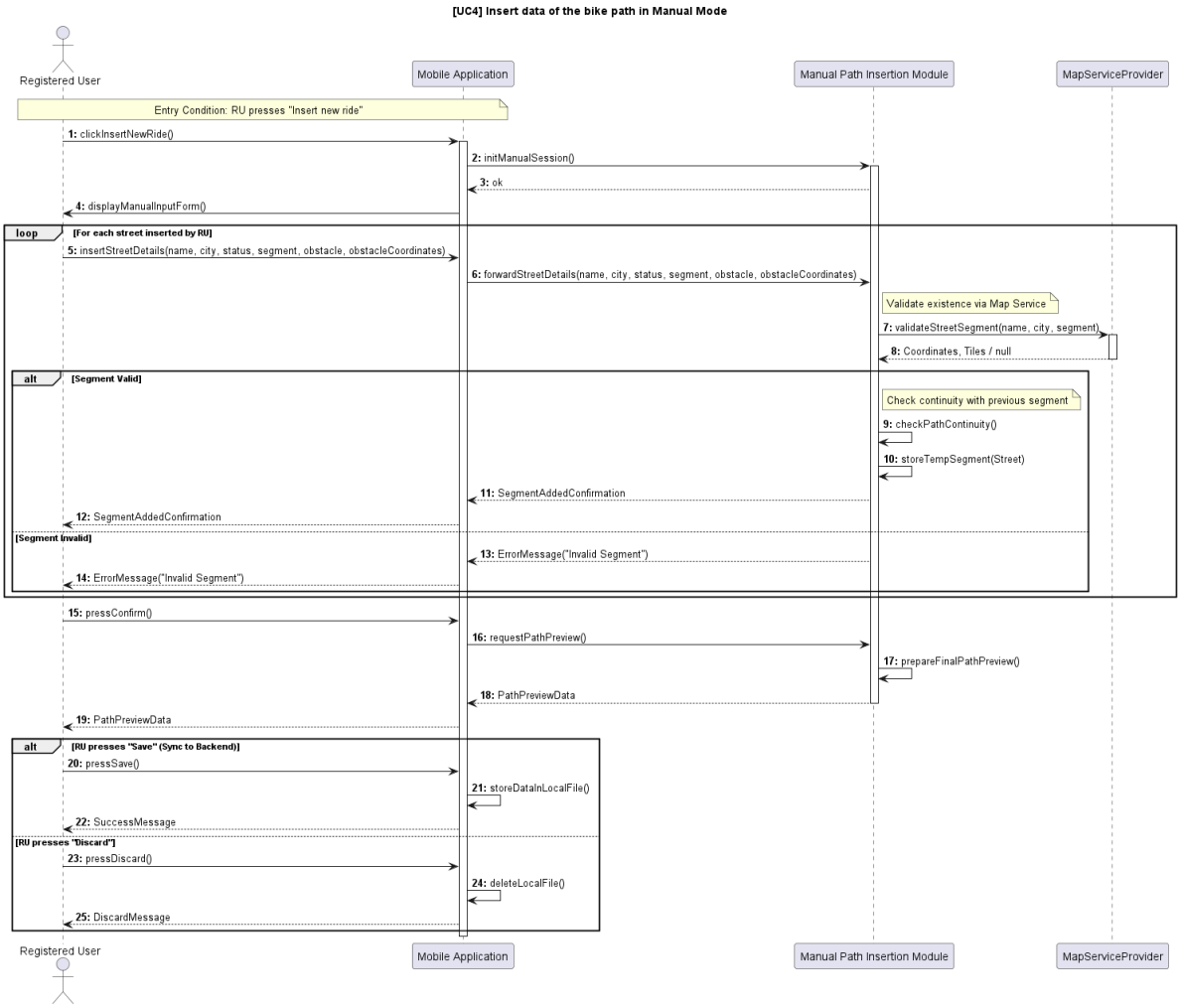



This diagram details the automated tracking process handled by the Mobile Application and its internal component, the Automated Path Data Collector (APDC).

Once the user initiates the ride, the APDC creates a local session and continuously records sensor data (GPS, Gyroscope, Accelerometer). The acquisition

loop terminates either through a manual user action or automatically if the system detects prolonged inactivity.

Following the stop event, the APDC performs parallel requests to the MeteoServiceProvider for weather conditions and the MapServiceProvider for map matching. The consolidated data is presented as a preview to the user. Finally, after any necessary manual corrections, the user decides to either save the session to local storage or discard the data entirely.



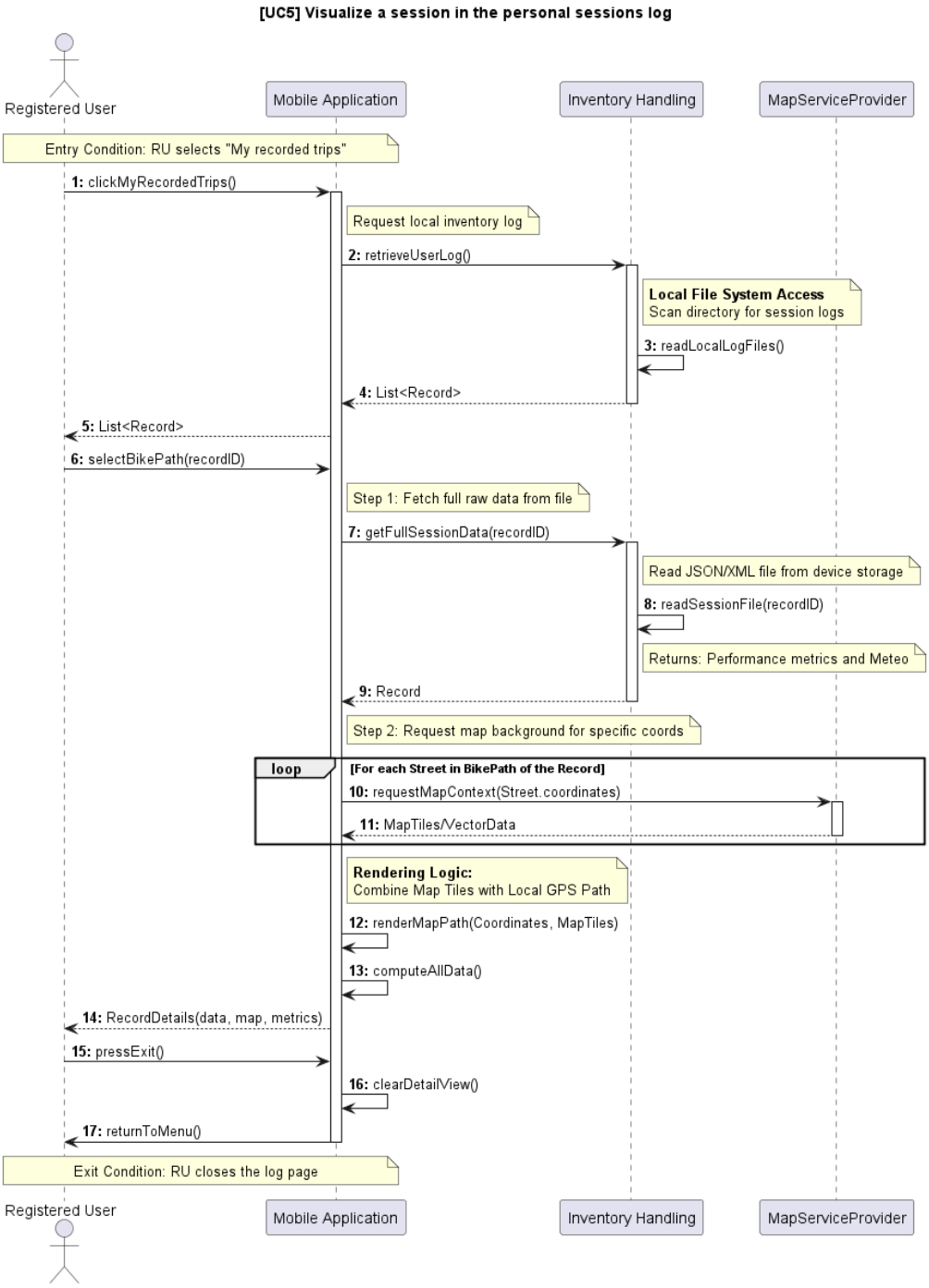
This diagram portrays the workflow for manually constructing a bike path, managed by the Mobile Application and the Manual Path Insertion Module (MPIM).

The process centers on an iterative loop where the user inputs details for individual street segments. For each segment, the MPIM validates the location against the MapServiceProvider and verifies geographical continuity with the previously added segment.

If valid: The segment is temporarily stored, and the user receives confirmation.

If invalid: An error message is returned.

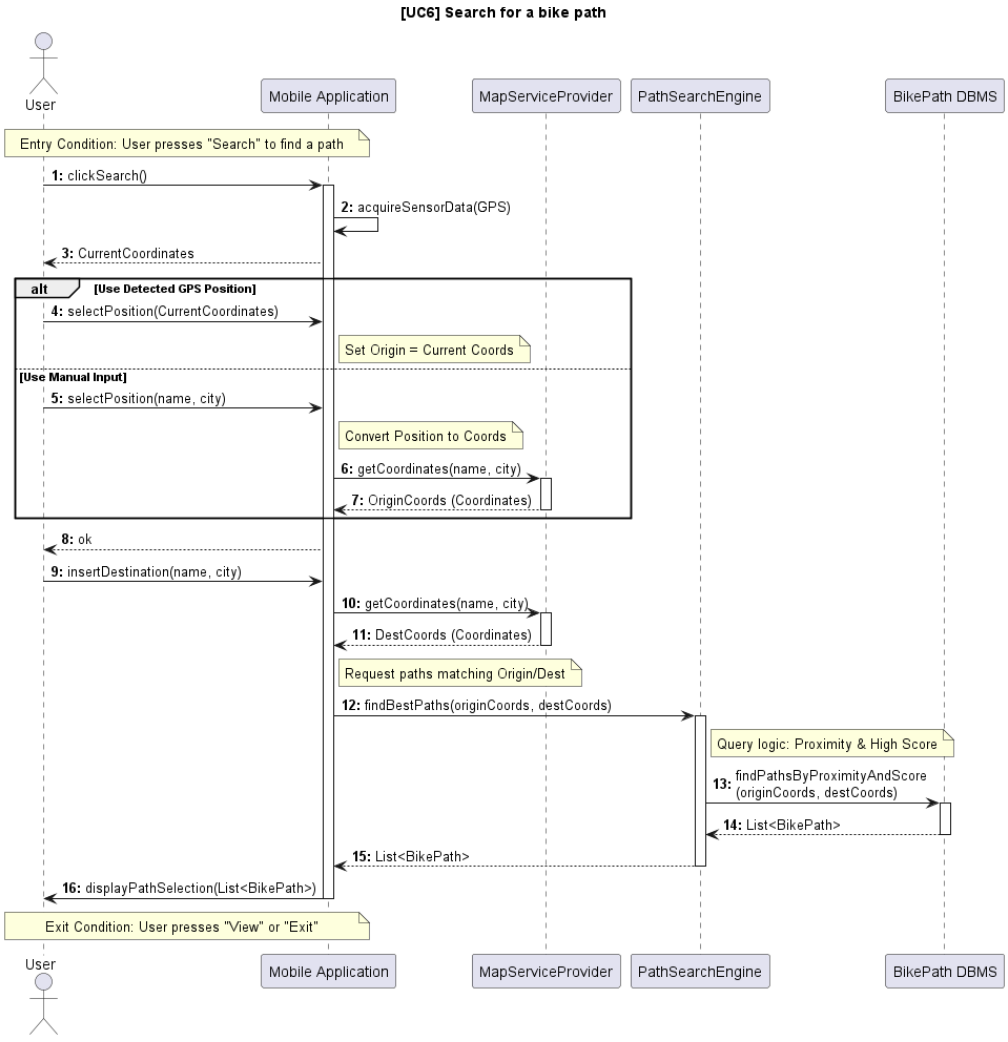
Once all segments are entered, the system generates a final path preview. The user then concludes the process by either saving the path to local storage or discarding the session.


[UC5] Visualize a session in the personal sessions log
Registered User
Mobile Application
Inventory Handling
MapServiceProvider
Entry Condition: RU selects "My recorded trips"
1: clickMyRecordedTrips()
Request local inventory log
2: retrieveUserLog()
Local File System Access
Scan directory for session logs
3: readLocalLogFiles()
4: List<Record>
5: List<Record>
6: selectBikePath(recordID)
Step 1: Fetch full raw data from file
7: getFullSessionData(recordID)
Read JSON/XML file from device storage
8: readSessionFile(recordID)
Returns: Performance metrics and Meteo
9: Record
Step 2: Request map background for specific coords
loop
[For each Street in BikePath of the Record]
10: requestMapContext(Street.coordinates)
11: MapTiles/VectorData
Rendering Logic:
Combine Map Tiles with Local GPS Path
12: renderMapPath(Coordinates, MapTiles)
13: computeAllData()
14: RecordDetails(data, map, metrics)
15: pressExit()
16: clearDetailView()
17: returnToMenu()
Exit Condition: RU closes the log page
Registered User
Mobile Application
Inventory Handling
MapServiceProvider

This diagram depicts the retrieval and visualization of a specific past ride. The process relies on the Inventory Handling component to access the device's local storage.

Upon accessing the log view, the Mobile Application requests the list of saved sessions from the Inventory Handling component, which scans the local file system and returns the available records.

Selection and Rendering: When the user selects a specific session, the application retrieves the full raw data (metrics and coordinates) from the local JSON/XML files. Map Integration: To provide visual context, the Mobile Application iteratively requests map tiles or vector data from the external MapServiceProvider corresponding to the recorded coordinates. Finally, it combines the local path data with the external map background to render the complete session details.

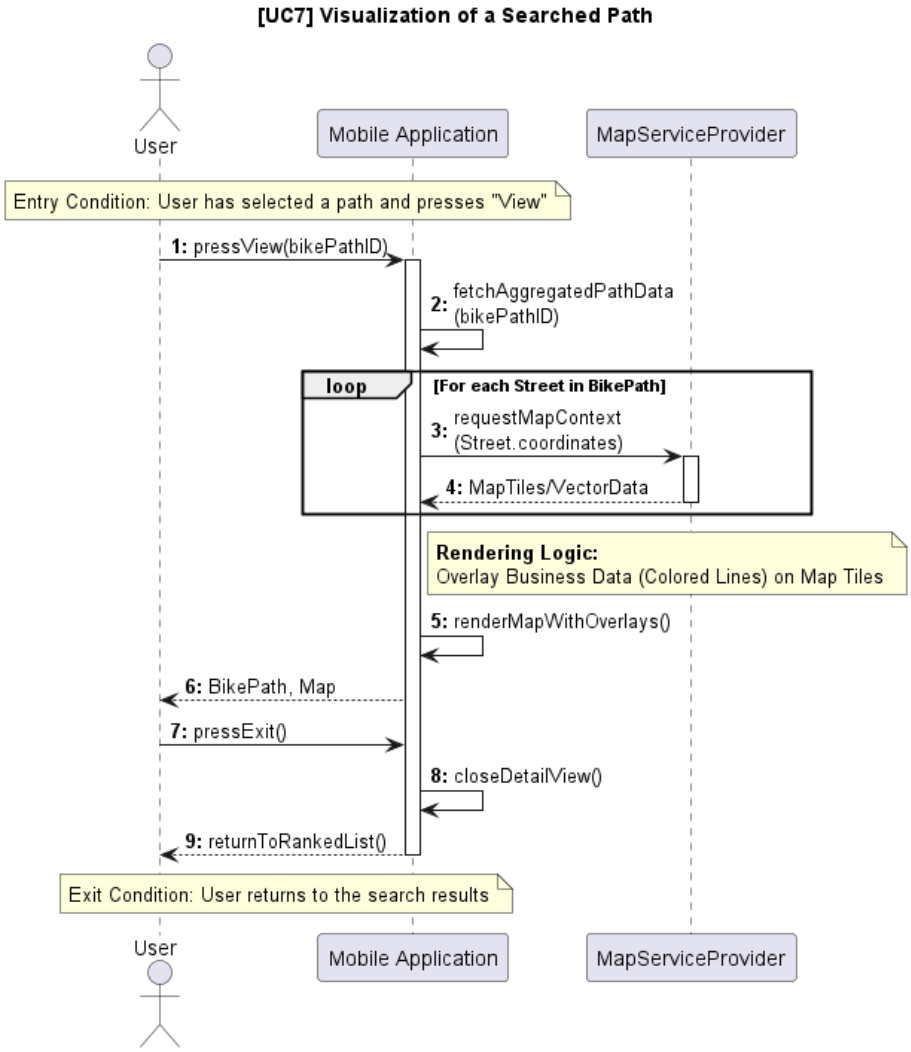



This diagram illustrates the search workflow managed by the Mobile Application and the backend PathSearchEngine.

The process begins with the definition of the trip's origin: the user may either select the automatically detected GPS position or manually input an address. In the case of manual input (and for the subsequent destination entry), the application invokes the MapServiceProvider to convert the addresses into geographical coordinates.

Once the origin and destination coordinates are established, the Mobile Appli-

cation queries the PathSearchEngine. This component accesses the BikePath DBMS to retrieve paths that match the specified proximity criteria, ranked by their quality score, and returns the result list to the user.

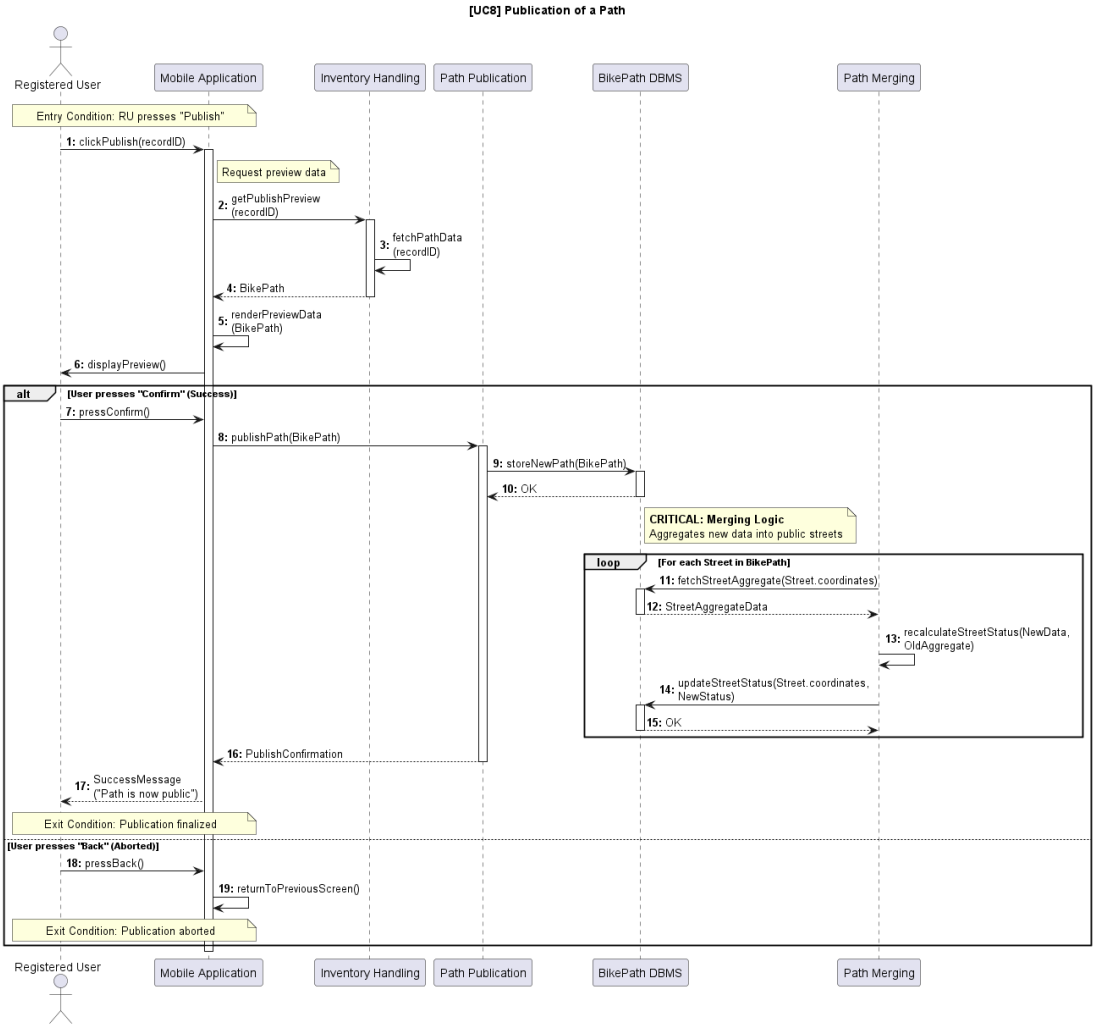



This diagram illustrates the visualization of a specific bike path selected from the search results.

It is explicitly assumed that the Mobile Application has locally cached the list

of paths retrieved during the previous Search phase (UC6). Consequently, the application retrieves the specific path data directly from its local memory, eliminating the need to query the PathSearchEngine or the backend database again for this step.

The only external communication required is with the MapServiceProvider: the application iteratively requests the necessary map tiles (or vector data) corresponding to the cached street coordinates. The final view is rendered by overlaying the local path data onto these retrieved map tiles.



This diagram depicts the process of transforming a private local record into a shared public path.

Initially, the Mobile Application retrieves the specific session data from the local Inventory Handling component to display a final preview to the user. Upon user confirmation, the data is transmitted to the backend PathPublication component.

The system first persists the new path in the BikePath DBMS. Subsequently, through the path merging service, it executes the critical merging logic: the component iterates through every street segment contained in the path, fetches the existing aggregate data for that specific location, recalculates the street's status based on the new input, and updates the global database. This ensures that the public map reflects the combined inputs of the community.

## 2.5 Component interfaces

In this section we provide details about the interfaces of the main components of the BBP system by specifying their methods, parameters, and return types.

### 2.5.1 Client subsystem

- **TrackSession**
  1. `initLocalSession(): void`
     Initializes a new tracking session and creates a temporary file on the device storage.
  2. `acquireSensors(GPS: GPSData, Gyro: GyroData, Accel: AccelData): void`
     Reads data from GPS, Gyroscope, and Accelerometer sensors.
  3. `appendToLocalFile(dataChunk: DataChunk): void`
     Appends the acquired sensor data chunk to the local temporary file.
  4. `stopAcquisition(): void`
     Stops the sensor data acquisition process.
  5. `analyzeMovementPatterns(): void`
     Analyzes sensor data to detect inactivity or movement patterns.
  6. `triggerAutoStop(): void`
     Automatically stops the session if inactivity is detected.
  7. `generateTripPreview(RefinedPath: RefinedPathGeometry, Weather: WeatherData): PreviewData`
     Generates a preview of the trip using refined path geometry and weather data.
- **ManageRecords**

1. `retrieveUserLog(): List<Record>`
   Retrieves the list of recorded sessions from the local storage.
2. `readLocalLogFiles(): List<Record>`
   Scans the local file system for session log files.
3. `getFullSessionData(sessionID: String): Record`
   Reads the full details (metrics, coordinates) of a specific session from the local file.
4. `readSessionFile(sessionID: String): Record`
   Helper method to read the content of a specific session file.
5. `getPublishPreview(recordID: String): BikePath`
   Retrieves the data of a session to be prepared for publication.
6. `fetchPathData(recordID: String): BikePath`
   Fetches the specific path data required for publication.

- **ManuallyInsertPath**

1. `initManualSession(): void`
   Initializes a new manual path insertion session.
2. `forwardStreetDetails(name: String, city: String, status: Status, segment: Segment, obstacle: ObstacleType, obstacleCoordinates: Coordinates): void`
   Receives street details entered by the user and forwards them for validation.
3. `checkPathContinuity(): boolean`
   Verifies if the new segment is geographically continuous with the previous one.
4. `storeTempSegment(SegmentData): void`
   Temporarily stores a valid street segment.
5. `requestPathPreview(): void`
   Requests the generation of the final path preview.
6. `prepareFinalPathPreview(): PathPreview`
   Aggregates all segments to create the final path preview.

- **StoreRecord**

1. `storeDataInLocalFile(): void`
   Persists the session or path data to a permanent local file.
2. `deleteLocalFile(): void`
   Deletes the temporary or unwanted local file.

- **MeteoProviderAPI**

1. `fetchWeather(GPS: Coordinates, Timestamp: DateTime): WeatherData`
   Fetches weather data for a specific location and time from an external provider.

- **MapProviderAPI**

  1. `requestMapMatching(RawGPSPoints: List<Coordinates>): RefinedPathGeometry`
     Requests map matching to refine raw GPS points into a path on the road network.
  2. `validateStreetSegment(name: String, city: String, segment: Segment): ValidationResult`
     Validates if a street segment exists and retrieves its coordinates.
  3. `requestMapContext(coordinates: List<Coordinates>): MapTiles`
     Retrieves map tiles or vector data for a set of coordinates.
  4. `getCoordinates(name: String, city: String): Coordinates`
     Geocodes an address (name and city) to coordinates.

- **PathDataTransformation**

  1. `computeStatusAndObstacles(GPS: GPSData, Gyro: GyroData, Accel: AccelData): (Status, List<Obstacle>)`
     Processes raw sensor data to determine the road status and identify obstacles.

#### 2.5.2 Backend system

- **ManageAccount**

  1. `registerUser(userDTO: UserDTO): void`
     Registers a new user in the system.
  2. `createRegisteredUserObject(): RegisteredUser`
     Creates a domain object representing the registered user.
  3. `authenticateUser(loginDTO: LoginDTO): AuthToken`
     Authenticates a user based on credentials.
  4. `createToken(user: RegisteredUser): AuthToken`
     Generates an authentication token for a verified user.

- **SearchPath**

  1. `findBestPaths(origin: Coordinates, dest: Coordinates): List<BikePath>`
     Finds paths matching the origin and destination, ranked by score.
  2. `rankAndConvertToSummary(paths: List<BikePath>): List<BikePath>`
     Ranks the found paths and converts them to a summary format for the list view.
  3. `getAggregatedPathDetails(pathID: String): BikePath`
     Retrieves the full aggregated details of a specific path.

- **PublishPath**

  1. `publishPath(path: BikePath): void`
     Publishes a new bike path to the system.

- **MergePath**

    1. `recalculateStreetStatus(newData: StreetData, oldAggregate: StreetAggregate): Status`
       Recalculates the aggregate status of a street based on new user input.

#### 2.5.3 Data system

- **AccessUserDB**

    1. `checkUserExistence(email: String, nickname: String): boolean`
       Checks if a user with the given email or nickname already exists.
    2. `createProfile(userEntity: User): void`
       Creates a new user profile in the database.
    3. `findUserByCredentials(email: String, nickname: String): User`
       Finds a user by their email/nickname and password hash.
    4. `updateLastLogin(userID: String): void`
       Updates the last login timestamp for a user.

- **AccessPathDB**

    1. `findPathsByProximityAndScore(origin: Coordinates, dest: Coordinates): List<BikePath>`
       Queries the database for paths near the origin/destination, ordered by score.
    2. `fetchAggregatedPathData(pathID: String): AggregatedPathData`
       Fetches the aggregated data for a specific path ID.
    3. `storeNewPath(path: BikePath): void`
       Stores a newly published path in the database.
    4. `fetchStreetAggregate(coordinates: Coordinates): StreetAggregateData`
       Retrieves the existing aggregate data for a specific street location.
    5. `updateStreetStatus(coordinates: Coordinates, newStatus: Status): void`
       Updates the status of a street in the database.

### 2.6 Selected architectural styles and patterns

**Client Server 4-tier Architecture**

The system adopts a 4-Tier Infrastructure. With this solution we can split the layer that manages the application and communicate with the Databases from the Gateway layer which hides all the application logic and acts as single entry point for all client requests. With this solution, the system exposes only a secure, monitored endpoint to the external network. The separation also allows the internal architecture to evolve or scale without impacting the client.

### Service-Based Architecture

The application is split into two independent Service: The Account Manager Service and the Path Manager Service. The first service is responsible for all the operations that regard the creation, modification and deletion of accounts. The second service is responsible for all the operation that regard the publication and the research of bike paths. This Separation of Concerns (both logical and physical) offers significant advantages:

- **Independent scalability:** The two services can be scaled separately from one another.
- **Fault Isolation:** Faults that affect one service do not affect the other.
- **Separation of Responsibilities:** Each service manages its own dedicated database schema, assuring decoupling.

### Bulkhead Pattern

The adoption of a Service-Based Architecture enables this pattern, responsible to ensure system resilience and graceful degradation. The pattern is implemented via the physical and logical separation of the Account Manager Service and the Path Manager Service. These two services use separate servers and different databases.

#### Command Pattern - CQRS

The system exploits the Command Pattern to decouple the actions requested to the system by a user and the implementation of that action in the server-side of the application. Since all the possible operations concern either the publication of a path or the search for a path we can leverage CQRS (Command Query Responsibility Segregation) to split the "Command" part from the "Query" part. The "write side" (command model) handles all the operations that regard the publication of paths prioritizing the consistency and rule validation for each published path while the "read side"(Query model) handles all the search operations. This query model will retrieve the pre-calculated bike path, ensuring quick response time due to the bypass of all complex domain logic.

#### Observer Pattern

The Observer pattern is used to manage the interaction between GPS and accelerometer and the user device. GPS and accelerometer are the 'Subject' and the ClientManager is the 'Observer'.

### 2.7 Other design decisions

#### Session Token

Each time a user logs into the system, they are provided with a TokenID that remains valid for a set amount of time. When a user is not registered or logged into the system they do not have this token. Without it, it is not possible to interact with the remote part of the application, which makes it impossible for the user to interact with the Path Manager Service to publish paths and to record personal Cycling Session. This implies that the user must be logged into the system to use these features. Without a token, a user can only search for published bike path.

#### Consistency Model

This choice has to do with the implementation of the databases. The system adopts linearizability for the account's databases, with this consistency model each operation related to accounts appears to take effect instantaneously at some point between its invocation and its competition. With this approach, even if multiple copies of the database exist, the system behaves as if there were only a single copy of the data, so the system ensures that, once a write is acknowledged by the system, all the future read operations (e.g. logins) will reflect that change, preventing race conditions that could lead to a bad user experience.
For the Bike paths' databases the system adopts an eventual consistent model prioritizing high availability and low-latency over consistency. Therefore, when a new path is published, the write is propagated asynchronously to all the other

replicas. When a user publishes a path this is written on the closest copy of the database, all the replicas eventually converge to the same state.

### Data Portability and Storage Strategy

Unlike public paths, personal records are not stored on the system's database but are kept exclusively on the user's device storage. This strategy reduces the overall cost of the infrastructure. To ensure that the user does not lose data if they uninstall the application, the system supports data export and import using GPX (GPS Exchange Format). The selected format is the standard for sharing GPS data and it ensures that the user can export personal rides and visualize them also on third party software or migrate them to a new device. This approach also reduces the impact of an eventual data breach.

# 3 USER INTERFACE DESIGN

This chapter illustrates the user interface mockups. Specifically, it revisits the mockups originally presented in the RASD, supplementing them with additional screens to provide a more design-oriented perspective.

Displayed below is the initial screen encountered upon launching the mobile application. As can be observed, this interface offers the option to log in (located at the top) or to proceed in Guest Mode (shown at the bottom). As previously stated, Guest Mode is restricted; consequently, users operating in this mode are limited solely to searching for paths within the BBP system database.

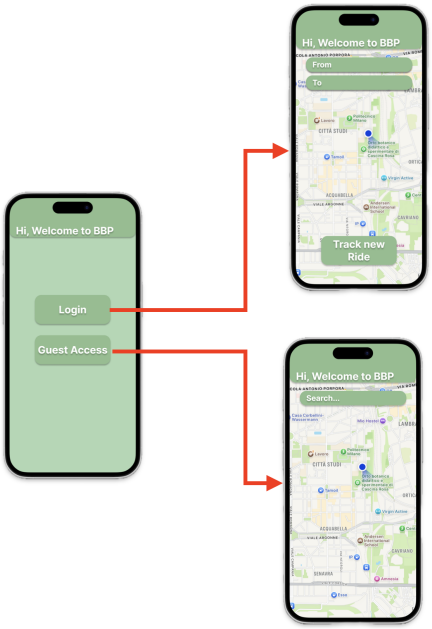



Figure 1: Homepage screen mockup

The following screens present the general workflow, illustrating the two tracking alternatives available for recording a new ride: automatic and manual. Specifically, upon clicking 'Track New Ride', the user is given the option to select the data acquisition mode.

If manual mode is selected (illustrated at the top), the user is presented with a form to sequentially enter each street (or street segment) and its corresponding status. Upon completing this process and confirming, the user is directed to a preview screen.

A similar screen is also displayed to users who, after selecting automated mode, decide to terminate the session (or when the system detects inactivity and automatically interrupts data acquisition).

In either scenarios, once on the preview screen, the user can further refine the manually entered or automatically detected data. In the second case, the refinement is mandatory. Finally, the user can choose to press 'Exit' to discard the collected data or 'Save' to store the acquired data locally. Should the latter option be selected, the user must be offered the choice to either publish the path or keep it private, making it viewable solely by them within their personal records log.

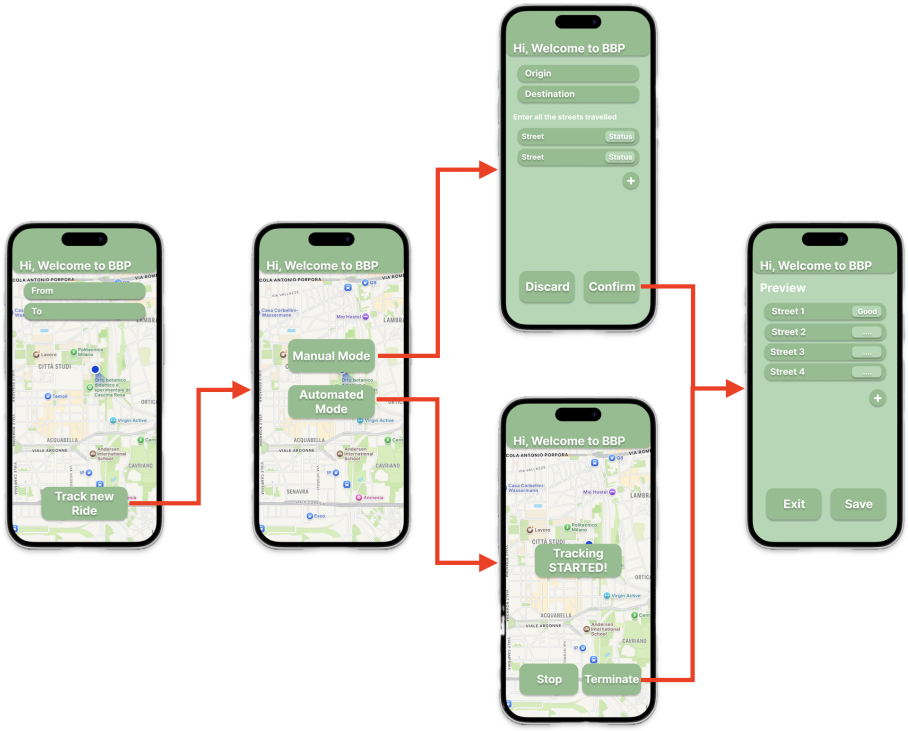



Figure 2: Tracking mode selection screen mockup

We now examine the automated path tracking workflow in greater detail. As can be observed, the first four screens are identical to those presented in the previous figure; indeed, the focus here is specifically on the final phase of the tracking process. In this stage, the user views the path preview and, after optionally refining the data, chooses to save the path, at which point they are presented with the option to immediately publish the record.

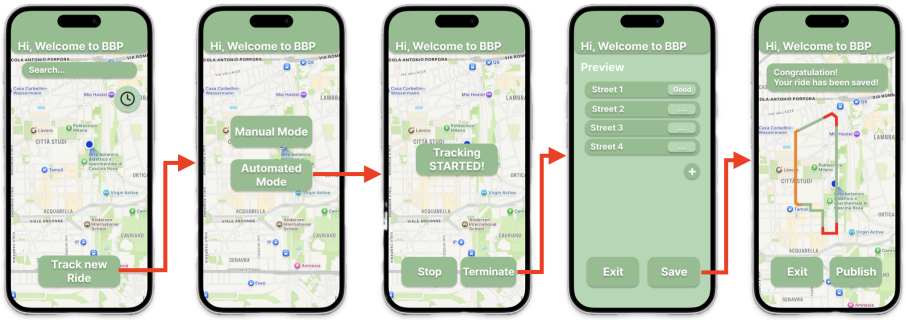



Figure 3: Automated tracking screen mockup

Proceeding further, we encounter the workflow for viewing a specific record within the sessions inventory. Specifically, the Home Screen features a button that grants access to the complete list of records. From this list, users can view the details of individual record, as depicted in the final screen on the right. This screen displays a hypothetical map that graphically highlights the path and indicates the status of its constituent streets.

This interface also allows users to review their performance metrics, which are obtained from the acquired data. Furthermore, within this view, the user must be provided with the option to modify the path's publication status (i.e., publishing a private path or reverting a public path to private if the user no longer wishes to share it) and, if necessary, to delete the record entirely.

It should be noted that these mockups are purely illustrative and may not explicitly depict every detailed of the described features, especially under the design point of view.

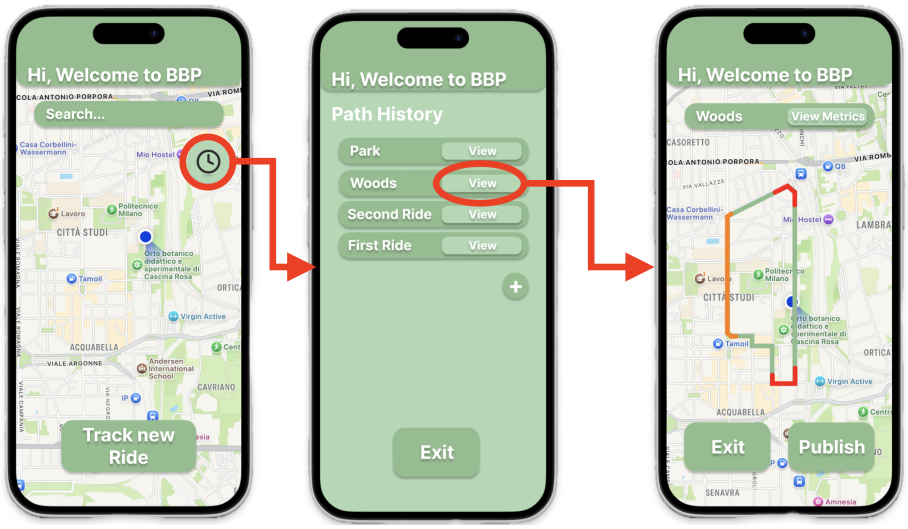



Figure 4: View recorded ride screen mockup

Finally, the last figure illustrates the path search workflow. As can be observed, the diagram deliberately depicts the scenario where the user is in Guest Mode. Consequently, starting from the main screen, the user clicks on 'Search,' enters the origin and destination, and is presented with a list of paths sharing similar origins and destinations, ranked by score.

From this list, the user has the option to press 'View' to examine the path's details and its associated score. It is important to note that the workflow is analogous for logged-in users. The primary distinction is that logged-in users, upon accessing the path detail screen, may be offered the option to immediately launch automatic tracking while being redirected to a third-party map application to get geographical directions; this is considered a useful but optional shortcut, and its inclusion is therefore left to the discretion of the implementer.




Figure 5: Search for a bike path screen mockup

It is repeated again for the sake of clarity that the mockups do not represent a strict set of guidelines to be followed in absolute terms during the implementation phase, but rather a starting point intended to clarify design choices and eliminate residual ambiguities.

# 4 REQUIREMENTS TRACEABILITY

In this section, we present the traceability matrix that maps the identified requirements to the corresponding system components responsible for their implementation. This matrix serves as a crucial tool for ensuring that all specified requirements are adequately addressed throughout the system's architecture and design. For the sake of clarity, we list below the requirements as they were defined in the RASD document, along with their identifiers:

- R1: BBP system allows users to create an account.
- R2: BBP system must distinguish and manage authorization for registered and not registered users.
- R3: BBP system shall enable a RU to initiate a trip tracking session.
- R4: BBP system shall enable a RU to pause a trip tracking session.
- R5: BBP system shall enable a RU to terminate a trip tracking session.
- R6: BBP system shall collect data from the mobile device's GPS, gyroscope and accelerometer.
- R7: BBP system shall geographically reconstruct the path that the user has ridden along from the GPS data that has been acquired in automated mode.
- R8: BBP system shall use the data that has been acquired in automated mode to signal the possible presence of obstacles.
- R9: In automated mode, BBP system shall monitor the biker's speed to detect the start, pause and end of the activity.
- R10: BBP system shall retrieve meteorological information from an external provider.
- R11: BBP system shall receive a manual confirmation from the user before publishing data that has been acquired in automated mode.
- R12: BBP system shall allow the RU to correct automatically identified data before publication.
- R13: In manual mode, BBP system shall retrieve and validate the names and status of the streets that form the path that the RU has ridden along.
- R14: BBP system shall publish the data that is manually submitted.
- R15: BBP system shall keep a private log of tracked sessions for each RU that is only accessible from that user.

- R16: BBP system shall compute and store the following performance metrics for a completed session: total distance covered, average speed, maximum speed, elevation gain, moving time and cadence.
- R17: for each published path, BBP system shall calculate the path's score.
- R18: the score of a path is computed by subtracting from the established maximum possible score a predefined number of points that depends on the individual streets' status and the relevance of the obstacles.
- R19: BBP system shall display paths that match the user's request in order of score.
- R20: BBP system shall display the existing paths whose origin and destination are within a predefined radius from the ones specified by the user.
- R21: BBP system shall allow registered users to edit the accuracy radius within a certain range for their path queries.
- R22: BBP system shall allow users to view all currently published path data, including their score.
- R23: if multiple registered users have inserted information about the same street in different paths, BBP system shall internally merge the information and only show the result of the merge.
- R24: the information merge performed by BBP shall consist of a computation that uses all the existing status values for the same streets and the insertion dates of the paths those streets are associated to.

Here is the traceability matrix:

| Requirements | Components |
|---|---|
| **Access and Users Management**<br>R1: Create account<br>R2: Distinguish/Manage authorization | User Management Service<br>Account Server<br>Mobile Application<br>Database Server for Registered Users |
| **Data Acquisition and Validation**<br>R3: Initiate trip tracking<br>R4: Pause trip tracking<br>R5: Terminate trip tracking<br>R6: Collect data (GPS, gyroscope, accelerometer)<br>R9: Monitor speed to detect activity | Automated Path Data Collection Service<br>Mobile Application |
| **Path reconstruction and metrics**<br>R7: Reconstruct path from GPS data<br>R8: Signal presence of obstacles<br>R16: Compute performance metrics | Path Data Transformation Service |
| **External Data**<br>R10: Retrieve meteorological information | Meteorological Data Retrieval Service |
| **Manual Input**<br>R13: Retrieve and validate manual street data | Manual Path Insertion Service<br>Mobile Application |
| **Local Data Management**<br>R12: Correct data before publication<br>R15: Keep private log of tracked sessions | Inventory Handling Service<br>Mobile Application |
| **Publication, Scoring & Merging**<br>R11: Manual confirmation before publish<br>R14: Publish manually submitted data<br>R17: Calculate path's score<br>R18: Score computation logic<br>R23: Merge information for same streets<br>R24: Information merge logic | Path Merging Service<br>Path Publication Service<br>Application Server<br>Mobile Application<br>Database Server for Published Paths |
| **Browsing and Search**<br>R19: Display paths in order of score<br>R20: Display paths within radius<br>R21: Edit accuracy radius<br>R22: View published path data | Path Research Service<br>Application Server<br>Mobile Application<br>Database Server for Published Paths |

# 5 IMPLEMENTATION, INTEGRATION AND TESTING

## 5.1 Overview

This chapter explains in details how the BBP system will be implemented and tested.

## 5.2 Implementation Plan

Since the BBP system is logically and physically separated into two distinct parts (online and offline functionalities) the two sub-system will be implemented independently of each other.

In order to facilitate rapid prototyping of core services, the token system that is used to give registered and logged users access to the reserved functionalities, will be initially stubbed. The logic will instead be implemented during the final integration phase.

- **Offline Functionalities**
  In order to implement offline functionalities, this part of the system, which is located in the Client tier, will be implemented by following a Thread-Based Strategy. Since the view of the local inventory and the export/import of local data depend on the existence of data, the implementation order of the various functionalities will be dictated by the following data lifecycle:

  Insertion → Visualization → Management

  At the implementation level, this implies that the order of development will be the following: first the functionalities related to path creation and deletion will be implemented, then those related to path visualization and finally the ones to perform the export and import of records.

- **Online Functionalities** The Server-side of the application will also be implemented by following a Thread-Based strategy.
  Since the online functionalities consist of two services (Account Manager Service and Path Manager Service) they can be implemented independently of each other. Two separate teams can work in parallel, one for the Account Management side and the other team for the Path Management side.

## 5.3 Component Integration and Testing

In this section, we will present the testing plan, which will follow the subsequent approach: the offline functionalities will be the first to be tested by using stubs when needed.

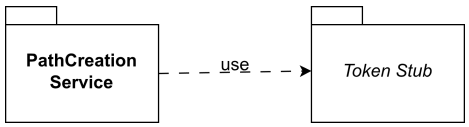
Online functionalities will be tested via drivers that represent a user to avoid the complexity introduced by the network, at least for the initial phases of testing.

Once both sections are individually tested, a full integration test (also considering the network) will be performed.

### 5.3.1 Testing Offline Functionalities

The following functionalities all require an active user session, i.e. a valid token. Nevertheless, in order to be coherent with what is said in the implementation plan, the testing phase will bypass the existent token verification mechanism, at least for the initial phases of testing.

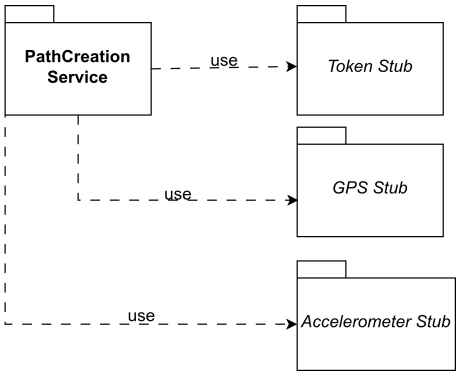
#### Manual Path Insertion




The tested feature include all the operation that let a user manually build a path by inserting the individual streets by hand. The test aims to verify the coherence of the inserted path. It verifies aspects such as:

- Rejecting street types tagged as unsuitable for bicycles (e.g., highways, pedestrian-only zones).
- Enforcing graph connectivity, preventing the insertion of a segment that is not physically connected to the previous one.
- The upper bound for number of possible streets in one path, and also that the system correctly manages this scenario.

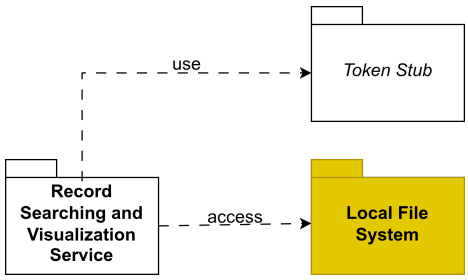
### Automatic Path Insertion




The tested functionalities include all those regarding the automatic tracking of a path. The system is stimulated with the injection of coordinates and other sensor data via stubs that simulates the real hardware systems that are used to automatically track a path. The test verifies aspects such as:

- The quantity of errors registered during the tracking does not significantly affect the registered path (less than 3 errors per kilometer). What we define as error is: the detection of an obstacle that do not exist, the detection of a wrong street, incorrect values regarding speed exceeding 5 km/h.
- The system reaction to the injection of completely wrong values such as: discontinuous coordinates, impossible values for elevation gain, average speed and other metrics.

### Path Visualization




The tested functionalities regard the ability to search and visualize local records and the specific tests aim to verify the correctness and soundness of the interaction between the system end the internal file system of the user's device.

#### Path exportation and Importation

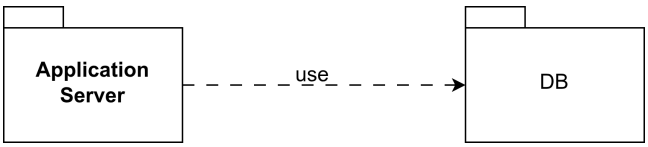
This test validates the system's ability to serialize internal records into the standard GPX format and, vice versa, to parse external GPX files into valid local records. The test verifies that:

- The file is syntactically correct and it can be also opened by third-party tools.
- All the internal attributes of a record (distance, speed, time) are correctly mapped and reported into the file.
- The system is resilient when stimulated with invalid or very large files.

### 5.3.2 Testing Online Functionalities

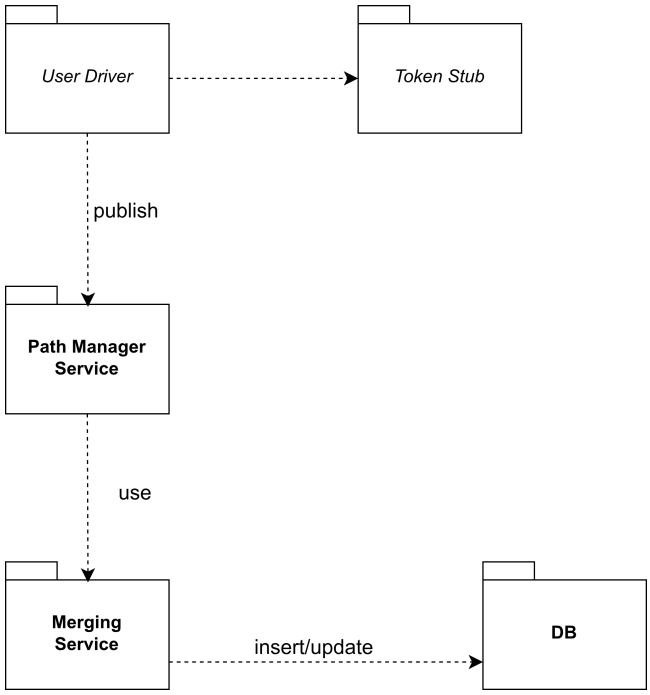
Online functionalities also require an active user session except for the ones related to the search of public bike paths.

#### Data Layer Test




This test aims to observe the interaction between the application logic and the database to verify the correctness of the Object-Relational Mapping configuration both for paths and accounts.

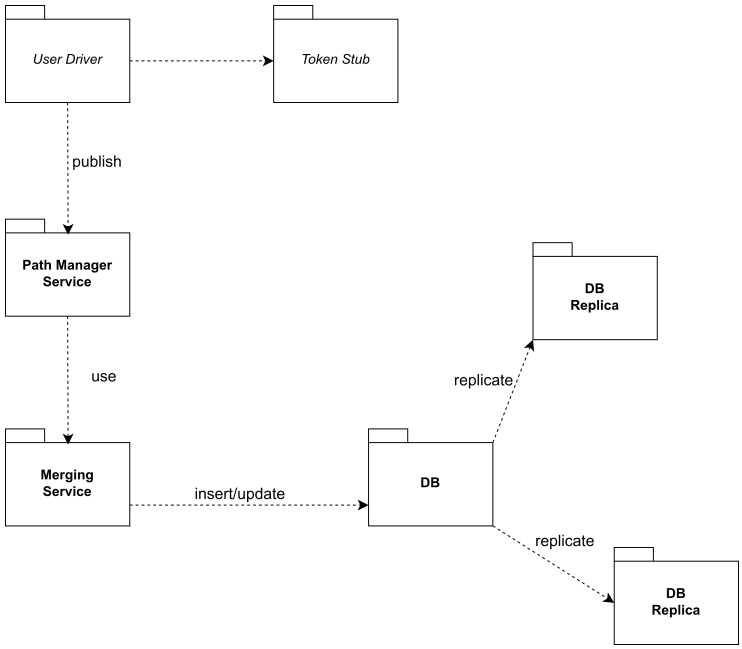
#### Path Merging




This test verifies that the merge mechanism works as intended in order to correctly update data related to streets and their status. The test aims to verify that:

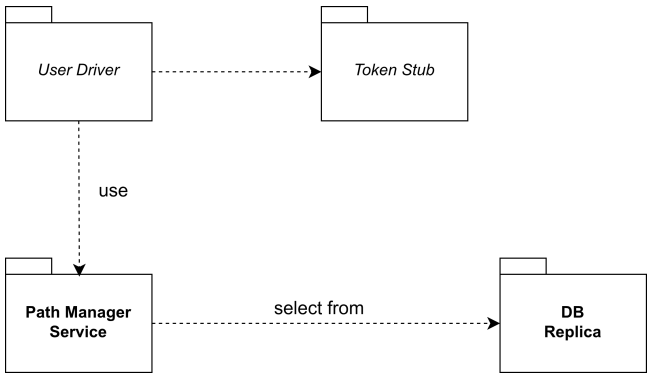
- Given a path P the Server is able to retrieve all the information about the roads that compose it from the database.
- For each road R in P, either the existent information is updated on the database or a new tuple is created (if the street has never been inserted).
- The overall score of the path is correctly computed according to the result of the merging.

**Path Publication**




This test aims to verify the correctness of the chosen consistency model. When a path is published, it should be replicated across all the DB replicas under 30 seconds.

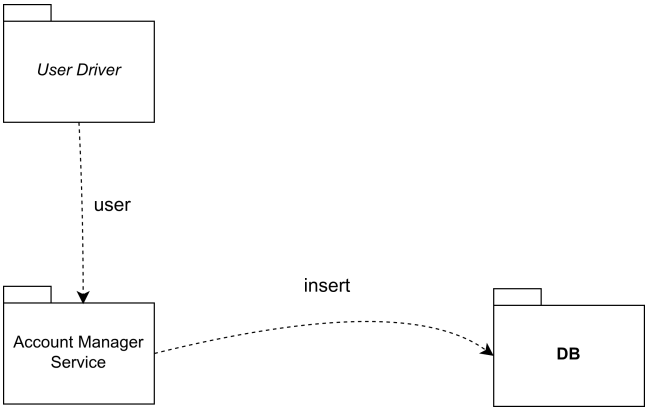
### Path search




The test aims to verify the correctness of the search logic, ensuring that the service correctly interprets the user queries and returns the paths correctly sorted

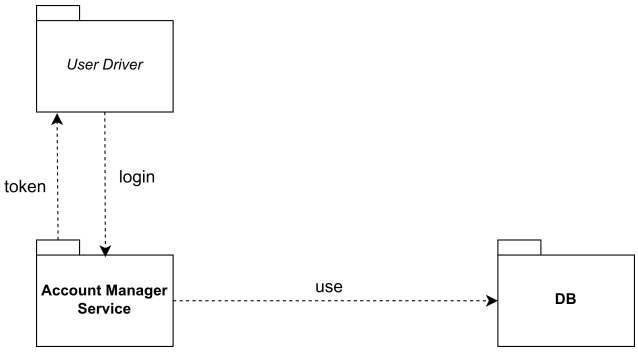
accordingly to their score.
The test must also verify the system's behavior when the result set is large. Since for paths the system follows an eventual consistency pattern, the test should be executed on a stable dataset in order to get deterministic results.

### Account Management - SignUp



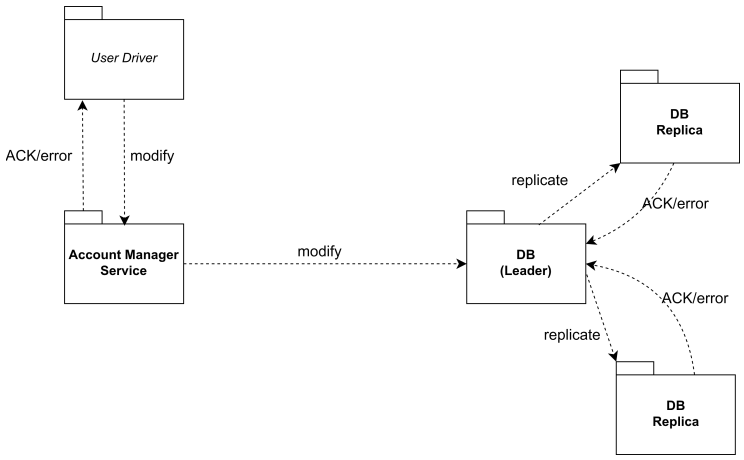

This test aims to verify that a new user entry is correctly added to the DB System when a new user signs up. In the followig sections, all tests related to account management are reported.

### Account Management - Login



This test aims to verify that a valid token is correctly delivered to a user when login is performed (if the user is registered).

#### Account Management - Data Modification




This test aims to verify that a logged user is able to modify their own information and that these edits are replicated accordingly to the chosen consistency model.

## 5.4 Additional Specifications on Testing

For every test suit that requires a database, a lightweight, disposable docker container which also runs the exact database engine that will be used in production will be used. After each test, the container is destroyed in order to ensure a clean state for future testing.

This approach is extended with container orchestration to simulate a scenario with multiple replica in order to verify the adopted consistency models.

# 6 EFFORT SPENT

The following table shows the approximate amount of hours that each group member spent on each chapter of this document. Please note that is calculation is only approximate, as each section still required active collaboration from all group members.

| Group Member | Chapter 1 | Chapter 2 | Chapter 3 | Chapter 4 | Chapter 5 |
|---|---|---|---|---|---|
| A. Rubino | 1 | 15 | 1 | 1 | 8 |
| E. Uvinsi | 1 | 20 | 5 | 3 | 1 |
| I. Vinciguerra | 2 | 10 | 1 | 1 | 1 |
| **TOT** | **4** | **45** | **7** | **5** | **10** |

# 7 REFERENCES


1. **Project documents**:
   - Specification document for the 2025/26 RASD project for the Software Engineering II course.
   - Slides on the WeBeep page of the Software Engineering II course.
2. **Standards and regulations**:
   - GDPR (General Data Protection Regulation).
3. **External service provider documentation/APIs**:
   - Documentation or specifications for the MSP.
   - Any specific API documentation or licensing terms mentioned for third-party software, libraries, or commercial mapping APIs.
4. **Technical specification documentation** for the following tools/protocols/formats:
   - Standard geospatial data formats (including GPX - GPS Exchange Format)
   - REST architectural style for web APIs
   - HTTP/1.1 and HTTPS
   - TCP/IP
   - TLS 1.3
   - JSON